

ТЕМА: ХЕШ



ВВЕДЕНИЕ

Хеш - многие слышали, но мало кто знает что это такое, тем более что такое хеш таблицы и функции, поэтому в этой теме мы расставим все точки над И

УРОК: ХЕШ-ТАБЛИЦА

Хеш-таблица

это структура данных, которая обеспечивает эффективный доступ к элементам с использованием хеш-функции для индексации данных. Она поддерживает операции вставки, поиска и удаления элементов в среднем за константное время



Принцип работы

Хеш-функция

Преобразует ключ в индекс массива, где будет храниться значение

Хеш-таблица

Массив, где элементы хранятся по индексам, вычисленным хеш-функцией

Основные операции

Вставка: вычисление индекса для ключа с помощью хеш-функции и сохранение значения в соответствующей ячейке массива

Поиск: вычисление индекса для ключа и поиск значения в массиве

Удаление: вычисление индекса для ключа и удаление значения из массива

Обработка коллизий: коллизия происходит, когда два ключа имеют одинаковый хеш-индекс

Для решения этой проблемы используются различные методы

Метод цепочек Chaining

Хранение нескольких элементов в одной ячейке массива в виде связного списка

Метод открытой адресации Open Addressing

Поиск следующей свободной ячейки в массиве по определённым правилам (линейное или квадратичное пробирование)

Пример реализации хеш-таблицы с использованием метода цепочек:

```
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                return
        self.table[index].append([key, value])

    def get(self, key):
        index = self.hash_function(key)
        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]
        return None

    def delete(self, key):
        index = self.hash_function(key)
        for i, pair in enumerate(self.table[index]):
            if pair[0] == key:
                del self.table[index][i]
                return
```

```
# Пример использования:  
ht = HashTable(10)  
ht.insert("name", "Alice")  
print(ht.get("name")) # Вывод: Alice  
ht.delete("name")  
print(ht.get("name")) # Вывод: None
```

Преимущества и недостатки

Преимущества

Высокая скорость доступа к элементам, эффективное использование памяти

Недостатки

Потенциальные проблемы с коллизиями и необходимость эффективного выбора хеш-функции и метода обработки коллизий

ВЫВОД

Мы рассмотрели основные принципы работы хеш-таблиц. Хеш-таблицы предоставляют эффективный способ хранения и доступа к данным. Понимание этой концепции поможет вам создавать более эффективные алгоритмы и структуры данных